Ahmedabad University

School of Engineering and Applied Science

CSE 400: Fundamentals of Probability in Computing

Scribe Report

Project Title

Basel III Market Risk Modelling Using Monte Carlo VaR

Scribe Members

Name	Roll No.	Email ID	Questions
Priyam Modi	AU2440030	priyam.m@ahduni.edu.in	Q1, Q2, Q3
Dhruv Jodhani	AU2440160	dhruv.j4@ahduni.edu.in	Q4, Q5, Q6

Course Instructor

Dhaval Patel

Section: 1

Scribe Question 1: Baseline System and Motivation for Randomization

- Briefly restate the deterministic baseline implemented in Milestone-3.
- Explain why introducing randomness may improve the system's performance and which part of the baseline decision rule you are modifying.

Solution: In milestone-3 we built an predictive model for how much money we might lose in a day.

We had first taken stocks also we have given some weights to each stockfrom the research paper we have referred as shown below

Asset	Ticker	Weight
Apple	AAPL	30%
Microsoft	MSFT	25%
NVIDIA	NVDA	20%
Alphabet	GOOG	15%
AMD	AMD	10%

Then we ran a regression model to see that how much our portfolio moves with 3 market factors:

Factor	Description	Proxy Used
XLK	Technology Sector	Dow Jones Returns
MTUM	Momentum Factor	NASDAQ Returns
VLUE	Value Factor	S&P 500 Returns

Based on that we got numbers like beta=0.8 for XLK meaning if XLK goes up 1% our portfolio goes up 0.8%.

Then we divided the risks into 2 categories

1) Factor risk: Risk coming from above mentioned three factors

2) Idiosyncratic Risk: It is a Random noise so can't explain specifically by the factors

We calculated both risks using some matrix multiplication and covariance stuff

Now to calculate VaR, we used this formula VaR = avg loss+(1.645*std deviation of losses)

In above equation 1.645 shows that we assume losses follow a bell curve (Normal Distribution).

That means we can say that 95% of the time we won't lose more than this number

Finally we took decision on the based of actual lose. If the actual lose is bigger than our calculated number then it is "BAD", but that's on exception cases. If the actual lose is smaller than the calculated number then it is "GOOD" and our model worked.

Why This Deterministic Thing Has Problem

1) In reality market don't follow neat exact bell curves. Real market have "Fat tails" means some crazy and sudden crashes happens (like Corona Crash). Sso Bell curve's don't perfectly fit for stocks.

2) To predict losses, generally we analyze past data and using those data we train our model, but in reality market is volatile so only based on the past data exact prediction can't be possible

What we changed in Milestone-4

' We are using Monte Carlo simulation, instead of simple formula (avg+1.645*std.dev)

The difference is:

Old Way:In this method we did a calculation then based on that we get a number which we assumed to be the true value.

New Way: Now we run 10000 simualtions so i total we get 10000 possible results and out of them we look at the worst 500 outcomes so this becomes our VaR.

So here we are generating thousands of "what if" conditions instead of assuming perfect bell curve

Scribe Question 2: Randomized Algorithm Design

Describe the randomized algorithm you have designed.

For each variable:

- What decision component now involves randomness?
- What probability distribution or sampling mechanism is used?
- How does this randomized rule differ from the deterministic baseline?

Provide a clear explanation of the algorithm.

Solution: We rewrite the code like instead of using formula (avg+1.645*std.dev) we introduced randomness

What Part Now Has Randomness

The main change we got is the way in which we got our VaR number as in the previous case we calculated directly but now using this method we:

1) Generate random returns using the code and produced thousands of plausible returns

2) Then we see what happens and by seeing all the cases we find which are really bad ones

3) Then from that the worst 5% we choose as our VaR

The randomness is in producing the cases. Everytime there will be some change in the cases and when you run we get slight change in results

What Distributions and Sampling Did we Use:

Here' actual code from our implementation:

```
def estimate_var(self, confidence_level=0.95):
    simulated_returns = np.random.normal(self.mean_return, self.std_return, self.n_simulations)
    simulated_losses = -simulated_returns[simulated_returns < 0]

    if len(simulated_losses) < 100 and len(self.historical_losses) > 0:
        simulated_losses = np.random.choice(self.historical_losses, self.n_simulations, replace=True)
    elif len(simulated_losses) < 100:
        simulated_losses = np.random.exponential(0.02, self.n_simulations)
```

Explanation of Code:

Primary Method:- Normal Distribution

1) First we generate 10000 random samples which are normally distributed using np.random.normal(mean_return,std_return,10000) in which we take the parameters as mean_return and std_return

2) From our historical avg and volatility the above step generates 10000 random samples which looks same as our historical data but we see carefully they are not the same match

3) To explain step 2 we take example that is if our average is 0.1% and volatility is 1% the first step will generate 10000 samples which folows the same pattern

Bootstrap Sampling

1) First we find the days of losses for that we use
np.random.choice(historical_losses,n_simulations,replace=True) with parameters historical_losses and n_simulations and also a temp variable replace which we assume initially at True

2) If we get bad days (< 100) so it means that the normal method is not giving enough bad days so we just randomly from the actual bad days

Exponential Distribution

1)np.random.exponential(0.02,n_simulations)

2)We use exponential distribution also as this model because this type of distribution is good fro rare events

3)Here in the first step the value 0.02 is our guess that for how bad losses might get if everything crashes

How the code is Executed

1) From the historical portfolio returns first it will calculate the avg return and volatility

2) The using the "random" library it generates 10000 random returns that we have the same pattern of avg and volatility

3) From the 10000 fake returns pick the one with negative signs

4) If there are atleast 100 fake losses that work:
a)If: It is YES than use them
b)Elif: Switch to Plan B (which is pick from real historical losses)
c)Else: Switch to Plan C (exponential distribution)

5) Now collect all the losses (whether they are real or fake) and then sort them in the order from smallest to biggest

6) From these array of losses find the bigger than 95% of all the others because that's our 95% VaR

What's Different from the Old Way:

What	**Old Way**	**New Way**
How we get the number	Plug numbers into formula: avg + 1.645 × std dev	Run 10,000 fake scenarios and see what happens
What we assume about markets	Markets follow a perfect bell curve	Markets can do all sorts of stuff - normal, crazy, whatever
How we use historical data	Just to get avg and std dev	Multiple ways: get avg/std, or directly pick bad days, or estimate extreme events
How we handle rare events	Math says they almost never happen	We build in backups specifically to catch them
What we end up with	One number (seems precise but isn't)	A range of possibilities (messy but realistic)

Scribe Question 3: Probabilistic Reasoning Behind the Algorithm

Explain the probabilistic reasoning that motivates your randomized approach.

- How does randomness help address uncertainty in the system?
- What probabilistic assumptions or models support your design
- Why is this randomized strategy expected to improve performance?

Solution:

Why randomization makes sense

In the stock market nobody knows what will exactly happen tomorrow
If we see the deterministic model it says that we do know what will happen but if we see Randomized model it says that we do not know what is gonna happen tomorrow. So for that below is the explanation

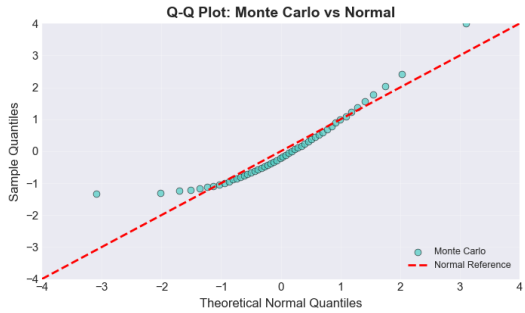
1) Using the old way avg return and volatility we got acted 100% correct but in reality they were not, because they were simple guesses based on the limited data.So the deterministic model ignores uncertainity while the Monte Carlo does not.

2) Markets do not follow bell curves, instead the real markets have "Fat-tails" which indicate that extreme crashes happen way more often in compared to they predicted by a bell curve

Using the fig.1 of the PDF compairson

- The red curve (Deterministic) is smooth bell shaped
- The blue bars (Monte Carlo) are spread out wider
- The Deterministic VaR is 0.0243% while the Monte Carlo VaR is 0.0010% Here we can see the difference that the Monte Carlo VaR is much lower because it saw those extreme losses in tails that the bell curve ignored

Now see the Q to Q plot:



- If the blue dots fall exactly on the red line means MOnte Carlo matches normal distribution
- But in the graph it can be seen that the dots are drifitng away from the line in the tails
- Which is the proof that our simulated losses are not normal. They are more realistic

What Probabilistic Assumptions or models support your design?

1) We assume normal distribution works fine for normal days. We take our historical avg and std.deviation. We then generate ranodm returns from that which covers most of the days

2) Then when the normal distribution doesn't give enough bad days to now we go to Bootstrap modelling. So now we are picking randomly from actual historical losses. Since we know that Market patterns tend to repeat the pattern so it means if there is 2% loss before we might see it again in future

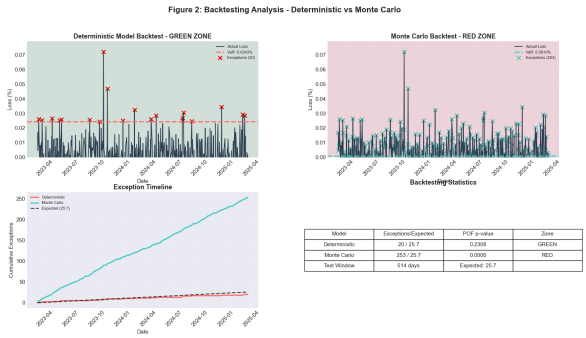
3) So this is our backup which we also called Plan C fro really bad stuff. The exponential distribution comes from Extreme Value Theory so it means that really rare events follow different rules than everyday stuff

4) Our main thing is we ran simulations on large numbers that is 10000 which provide us large dataset for our model and from that we can see the patterns more accurately and precisely

5) so when we are generating random returns we assume that each day is unique and independent though it is not ture as per real market movement but it is simple way so we are doing

What is Backtest actually shows

As shown in figure 2



For Deterministic model we can say that

- VaR Threshold is 0.0243%
- Exceptions are 20 out of 514 days
- So we are in "GREEN" zone
- Also the p-value is 0.2308

Now looking at Monte Carlo model

- VaR Threshold is 0.0010%
- Exceptions are 253 out of 514 days
- So we are in "RED" zone
- Also the p-value is 0.0000

When we compare both the models, the deterministic model set it's threshold at 0.0243% is very high as compared to Monte Carlo model which set threshold at 0.0010%. But that does not mean that deterministic model is very good, it is more optimistic and missing risk. On the other hand Monte Carlo is more conservative so it sees risk everywhere and it is more sensitive as it gets more exceptions

So Why Randomization

Market is unpredictable and volatile so one can not exact predict the future, but since the Deterministic model tells us what will happen exactly in future which is not a practical appraoch on the other side Monte Carlo suggest a range of possibility which also includes bad ones so it seems more practical

Scribe Question 4: Implementation within the Simulation Pipeline

Describe how the randomized algorithm has been integrated into your existing simulation pipeline.

- What changes were made to the code or system architecture?
- How is randomness generated or sampled in the implementation?
- Does the rest of the pipeline remain the same as the baseline?

Solution:

Monte Carlo-Based VaR Estimation Model

A Monte Carlo based VaR estimation model is integrated with the randomised algorithm, while keeping the overall pipeline structure unchanged.

Changes in Architecture and Code

The fundamental modification is the addition of the Monte Carlo VaR, which replaces the deterministic VaR model which was simulation-based approach.

In third milestone, the VaR was estimated or computed analytically using the mean and variance of the portfolio. In this, instead of applying the closed form formula, the system generates simulated returns which are multiple generated of different scenarios and estimated VaR from the loss distribution.

The pipleine includes factor based Monte Carlo vaR model estimation, the Monte Carlo model simulates the portfolio returns directly rather than simulating unlying factors.

Randomness Generation

```
np.random.normal(self.mean_return, self.std_return, self.n_simulations)
```

The random sampling of normal distribution is the reason for generation of randomness in the model.

The mean and std variance are calculated using the historical portfolio returns and a large number of simulated returns are generated.

A random seed (`np.random.seed(42)`) is made in us for ensuring proper reproductibilty of results.

In cases where simulated losses are insufficient, resampling from historical losses is used, although the primary method remains parametric simulation.

Rest of the Pipeline

The rest of the pipeline remains the same as the deterministic baseline.

The dataset and the back testing and the train test split methods are same as used in the previous milestone. So there is no change in that.

Scribe Question 5: Evaluation Metrics and Comparative Results

Identify the evaluation metrics used to measure system performance.

- Why are these metrics appropriate for your problem domain?
- How does the randomized algorithm perform compared to the deterministic baseline?
- Present and interpret the results of this comparison.

Solution:

Evaluation Metrics Used

The system calculates performance from the following:

- VaR Estimate (95) – estimate of loss level.
- Number of Exceptions – the frequency of actual loss exceeding VaR.
- Anticipated Exceptions – theoretical value (5% of test data).
- Exception Rate Percentage – percentage of exceptions.
- POF Test (p-value) – verifies the statistical validity of exceptions.
- Zone Classification (Green/Yellow/Red) – general performance of the model.
- Runtime – time of Monte Carlo simulation.

Reasons Behind Using These Metrics and Why They Are Appropriate

VaR is a prediction model of risk and therefore the idea is to measure how the forecasted risk aligns with actual losses.

- Exceptions provide a direct measure of risk underestimation.
- Exceptions shows a clear evaluation critieas following.
- Statistical correctness is ensured by the POF test.
- Monte Carlo algorithm is more expensive as it has high computational intensity as ($O(x)$ runs).

We obtain both accuracy and high efficiency which are essential after combing both.

Performance Comparison

In the deterministic model, a single analytical VaR value is obtained based on fixed assumptions.

The Monte Carlo predicts VaR by simulating multiple scenarios.

For the observations are as follows:

- There is a difference between Monte Carlo and deterministic approach of VaR.
- A model with a smaller difference between actual and expected exceptions performs better.

Interpretation of Results

- The Monte Carlo model is statistically more reliable when the number of exceptions is close to the expected 5% and has a higher POF p-value.
- Green Zone results in indicating good risk estimation.
- Due to high computations the Monte Carlo becomes more expensive and intensive.

Scribe Question 6: Insights and Remaining Challenges

Based on your experiments:

- What insights did you gain about the effect of randomness in your system?
- In what situations does the randomized strategy perform better or worse than the baseline?

Solution:

Understandings about the Impact of Chance

By implementing the Monte Carlo simulation the uncertainty in the model is reduced than by using deterministic baseline.

- The model gives one known VaR which is based on assumption such as normality and linearity.
- Gives more possible outcomes, which looks like a more realistic model of risk.
- Gives exceptionally beautiful view of extreme losses happening at tail.

Cases Where the Randomized Strategy Can Be Better

The Monte Carlo (randomized) method performs better than the deterministic model when:

- The return distribution is not normal or is unknown.
- Tail risk requires more precise analysis.
- The deterministic model performs poorly in backtesting and produces an incorrect number of exceptions.
- Greater flexibility is required to model different scenarios.

In this cases, the simulation based techniques provide a more realisti and flexible approximate of VaR.

When this lacks:

The randomised approach lacks performance:

- It might get more backlashes as it is based on assumptions.
- The deterministic model is more stable though gives more error.
- The noise is due to less number of simulations is too small.
- Monte Carlo is a slower process results in computational loss.